Day 3 Unguided Session Assignment

Vector Mathematics: The Internals, Part 2

Participant: Gowtham | Repository: vector-math-lab (cosine-vector-mathematics) | 31 October 2026

Submission checklist

Item	Done	Where
Part 1 (Participant Guide): E1-E4, checker, step 4	Yes	Part 1 section
Task A: three pairs, prediction recorded before calculating, working shown	Yes	Task A section below
Task B: nearest() in my_vector_math.py, tests, checker 4/4 PASS	Yes	my_vector_math.py, tests/test_mine.py
Task B: prompt log with at least one thing challenged or fixed	Yes (3 fixes)	PROMPT_LOG.md
Task C: data/my_scenario.json, scenario_matrix output, prediction note	Yes	Task C section; evidence/
Task D: decision table	Yes	Task D section
Task E: false-similarity example + explanations	Yes	Task E section
Stretch tasks (identity, centroid, sensitivity)	Yes (all three)	Stretch section
Synthetic data only; no API keys, .env or real bank data	Yes	All items invented; .gitignore fixed

Tests: **48/48 pass** (python -m unittest discover -s tests). Checker: **4/4 PASS** (python step3_check_my_code.py).

Part 1 (guided) - from the Participant Guide

The Participant Guide is the Part 1 guided lab; the Assignment Outline is Part 2. Both are covered in this one file. Part 1 results:

Exercise	Dot product	Cosine	Euclidean	Matches answer key?
E1 [3, 4] and [4, 3]	24	0.960	1.414	Yes
E2 [3, 4] and [6, 8]	50	1.000	5.000	Yes
E3 loan and mortgage	34	0.972	1.414	Yes
E4 loan and fraud	11	0.344	6.481	Yes

Step 2-3: my_vector_math.py was written with Claude Code (Prompt 1) and checked: the checker went from 0/4 (TODO) to 4/4 PASS. Details are in Task B.
Step 4 (why cosine), the key result: for the query "Question about our loans and customer service", cosine ranks the very long annual report first (0.998), but distance ranks it far away (5.916) because its vector is much longer. Cosine and distance disagree when lengths differ. For text, cosine is the safer default, and documents are cut into similar-sized chunks.

One sentence each. Dot product: multiply matching numbers and add, which is large when both are strong in the same places. Length: the square root of the sum of squares, i.e. how long the arrow is. Cosine: the dot product divided by both lengths, which tells whether the arrows point the same way. Distance: the straight-line gap between the two end points, where lower is more similar. Cosine ignores length because dividing by both lengths leaves only the direction.

Task A - Calculate by hand again

My prediction, written BEFORE calculating: 1 = KYC and AML (both compliance topics), 2 = overdraft and chargeback (both money disputes on an account), 3 = complaint and fraud (service vs risk).

Pair	Dot product	Cosine similarity	Euclidean distance	My prediction (rank, 1 = most similar)
overdraft and chargeback	22	0.773	3.606	2
KYC and AML	24	0.802	3.464	1
complaint and fraud	10	0.358	6.000	3

Working

```
overdraft = [4, 2, 1, 3]   chargeback = [1, 4, 1, 3]
  dot  = 4x1 + 2x4 + 1x1 + 3x3 = 4 + 8 + 1 + 9 = 22
  |o|  = sqrt(16 + 4 + 1 + 9) = sqrt(30) = 5.477      |c| = sqrt(1 + 16 + 1 + 9) = sqrt(27) = 5.196
  cos  = 22 / (5.477 x 5.196) = 22 / 28.460 = 0.773
  dist = sqrt(3^2 + (-2)^2 + 0^2 + 0^2) = sqrt(13) = 3.606

KYC = [1, 1, 5, 1]   AML = [0, 4, 4, 0]
  dot  = 0 + 4 + 20 + 0 = 24
  |K|  = sqrt(1 + 1 + 25 + 1) = sqrt(28) = 5.292      |A| = sqrt(0 + 16 + 16 + 0) = sqrt(32) = 5.657
  cos  = 24 / (5.292 x 5.657) = 24 / 29.933 = 0.802
  dist = sqrt(1^2 + (-3)^2 + 1^2 + 1^2) = sqrt(12) = 3.464

complaint = [0, 1, 0, 5]   fraud = [0, 5, 2, 1]
  dot  = 0 + 5 + 0 + 5 = 10
  |c|  = sqrt(0 + 1 + 0 + 25) = sqrt(26) = 5.099      |f| = sqrt(0 + 25 + 4 + 1) = sqrt(30) = 5.477
  cos  = 10 / (5.099 x 5.477) = 10 / 27.928 = 0.358
  dist = sqrt(0^2 + (-4)^2 + (-2)^2 + 4^2) = sqrt(36) = 6.000
```

Checked afterwards with python vector_math.py <word1> <word2>: all nine numbers match (evidence/taskA_vector_math_check.txt).

Reflection. My ranking was right, but I expected KYC and AML to be far ahead, and the numbers show they are only just more similar than overdraft and chargeback (0.802 vs 0.773). The numbers also showed something intuition missed: AML is mostly fraud_risk plus KYC, so it shares a lot with fraud. Complaint and fraud are not completely unrelated (0.358), because both have some fraud_risk and service weight.

Task B - Extend the code with Claude Code

I ran the prompts from CLAUDE_CODE_PROMPTS.md through the Claude Code CLI (model sonnet) inside the lab folder. Prompt 1 (Part 1) came first, because my_vector_math.py was still the placeholder file and the checker showed 0/4 (TODO). I then read every line Claude Code produced and challenged it.

Final nearest()

```
def nearest(query: list[float], candidates: dict[str, list[float]], metric: str = "cosine") ->
list[str]:
    """Return candidate names sorted from most to least similar to the query.

    candidates maps each name to its vector. metric is "cosine" (higher is better)
    or "euclidean" (lower is better). Equal scores keep the input order.
    Raises ValueError for an unknown metric, an empty candidate dict, a zero vector
    (cosine only) or vectors of different lengths - naming the candidate at fault.
    """
    # Pick the scoring function and the sort direction for the chosen metric.
    if metric == "cosine":
        score = cosine_similarity
        # For cosine, a higher score means more similar, so sort descending.
        descending = True
    elif metric == "euclidean":
        score = euclidean_distance
        # For euclidean, a lower distance means more similar, so sort ascending.
        descending = False
    else:
        raise ValueError("metric must be 'cosine' or 'euclidean'")
    # MY FIX 1: an empty dict used to return [] silently; a caller doing result[0] then crashed far
away.
    if not candidates:
        raise ValueError("candidates is empty - nothing to compare the query with")
    # Score every candidate against the query.
    scores = {}
    for name, vec in candidates.items():
        try:
            # MY FIX 2: round to 12 decimals. Floating-point noise made equal cosines differ
            # (e.g. [1,1,1] vs [2,2,2] -> 1.0000000000000002 but vs [7,7,7] -> 1.0), which broke
            # the "ties keep input order" promise and let a later candidate jump ahead.
            scores[name] = round(score(query, vec), 12)
        except ValueError as exc:
            # MY FIX 3: say WHICH candidate is bad, so one broken record is easy to find.
            raise ValueError(f"candidate {name!r}: {exc}") from exc
    # Sort names by score (sorted is stable, so ties keep their input order).
    return sorted(scores, key=scores.get, reverse=descending)
```

What I found and fixed

Issue	Evidence	Fix
Ties were NOT kept in input order (Claude claimed they were)	Query [1,1,1]: [7,7,7] scores 1.0, [2,2,2] scores 1.0000000000000002. Both point the same way, yet B jumped ahead of A (evidence/taskB_issue_probe.txt)	Round scores to 12 decimals before sorting, so true ties really are ties and the stable sort keeps input order
Empty candidate list returned [] silently	nearest([1,2], {}) -> []. A caller using result[0] would crash later with an unclear IndexError	Raise ValueError("candidates is empty ...")
Error did not name the bad record	One zero-vector candidate stopped the whole search with "Cosine similarity is undefined for a zero vector"	Re-raise as "candidate 'empty_record': ..."
A tie test that could not fail	Claude's tie tests used [1,0] and [2,0], which give exactly 1.0	Added test_float_noise_does_not_break_a_true_tie
Repo test broke once the stub was implemented	test_stub_reports_todo expected TODO from the real file	Simulate the stub with mock.patch, so the test checks the checker, not the state of my file

Proof that my tests matter: run against Claude's original nearest(), my updated tests/test_mine.py gives 4 failures. With my fixes: 48/48 tests pass and step3_check_my_code.py is still 4/4 PASS.

```
Checking my_vector_math.py against the answer key

  dot                 PASS
  norm                PASS
  cosine_similarity   PASS
  euclidean_distance  PASS

4/4 functions pass.
All good. Now explain to your partner, in your own words, what each function does and why.
```

Prompt log

#	Prompt	Claude Code produced	Wrong / missing -> what I changed
1	Prompt 1 (verbatim): write dot, norm, cosine_similarity, euclidean_distance	Four correct functions with comments; did not run the checker itself	Maths correct (4/4 PASS, matches my hand answers). It mixed LF line endings into a Windows file, which I normalised.
2	Prompt 5 (verbatim): add nearest()	dict-based nearest(), cosine desc / euclidean asc, ValueError for unknown metric	False tie claim, silent [] for empty input, unnamed bad candidate -> fixes 1-3 above
3	Prompt 4 adapted: unittest tests for nearest() incl. edge cases, in tests/test_mine.py	30 passing tests; flagged that test_stub_reports_todo now fails	Tie tests could not fail; two tests locked in the silent []. Replaced 2, added 4, fixed the repo test.

Claude Code cost for the three prompts: about $0.26. The full log is in PROMPT_LOG.md.

Task C - My scenario table

Scenario: 2. Policy & product knowledge search. Six short synthetic policy snippets that an agent's question could be matched against. Dimensions: eligibility, fees_and_charges, account_lifecycle, customer_protection (0-5).

Item (synthetic)	eligibility	fees	lifecycle	protection
Eligibility: age 21-60 with salary account	5	0	2	0
Fees: annual fee and late-payment charge	1	5	1	1
Early closure: 2% charge within 12 months	0	4	5	1
Dispute: report unauthorised card use in 60 days	0	2	1	5
Privacy: how customer data is used and shared	1	0	1	4
Complaints: escalate to ombudsman after 30 days	0	1	1	5

Prediction (written before running)

```
Task C - predictions written BEFORE running scenario_matrix.py
Scenario: 2. Policy & product knowledge search
Most similar pair (prediction):  Dispute  <->  Complaint handling  (both about customer protection)
Least similar pair (prediction): Eligibility  <->  Complaint handling  (who can apply vs what to do
when unhappy)
Pair that may surprise me:       Fees  <->  Early closure  (early closure is mostly about a CHARGE,
so it may score closer to Fees than I expect)
```

Output of python scenario_matrix.py data/my_scenario.json

```
Dimensions: ['eligibility', 'fees_and_charges', 'account_lifecycle', 'customer_protection']

COSINE SIMILARITY (1 = same direction)
                                                1      2      3      4      5      6
1. Eligibility: age 21-60 with salary account        1.00   0.25   0.29   0.07   0.31   0.07
2. Fees: annual fee and late-payment charge          0.25   1.00   0.76   0.55   0.27   0.40
3. Early closure: 2% charge within 12 months         0.29   0.76   1.00   0.51   0.33   0.42
4. Dispute: report unauthorised card use in 60 days   0.07   0.55   0.51   1.00   0.90   0.98
5. Privacy: how customer data is used and shared     0.31   0.27   0.33   0.90   1.00   0.95
6. Complaints: escalate to ombudsman after 30 days   0.07   0.40   0.42   0.98   0.95   1.00

EUCLIDEAN DISTANCE (0 = identical)
                                                1      2      3      4      5      6
1. Eligibility: age 21-60 with salary account        0.00   6.56   7.14   7.42   5.74   7.21
2. Fees: annual fee and late-payment charge          6.56   0.00   4.24   5.10   5.83   5.74
3. Early closure: 2% charge within 12 months         7.14   4.24   0.00   6.00   6.48   6.40
4. Dispute: report unauthorised card use in 60 days   7.42   5.10   6.00   0.00   2.45   1.00
5. Privacy: how customer data is used and shared     5.74   5.83   6.48   2.45   0.00   1.73
6. Complaints: escalate to ombudsman after 30 days   7.21   5.74   6.40   1.00   1.73   0.00

Most similar pair  : Dispute: report unauthorised card use in 60 days  <->  Complaints: escalate to
ombudsman after 30 days   (cosine 0.98)
Least similar pair : Eligibility: age 21-60 with salary account  <->  Dispute: report unauthorised
card use in 60 days   (cosine 0.07)
Was this what you predicted? If not, which of your numbers surprised you?
```

Prediction vs result

- **Most similar: correct.** Dispute <-> Complaints, cosine 0.98, distance 1.00.
- **Least similar: almost correct.** Eligibility <-> Dispute scored 0.068, just below my pick, Eligibility <-> Complaints (0.072). Both pairs share only one small number.
- **Surprise:** Fees <-> Early closure was 0.76, as I suspected. The real surprise was Privacy <-> Dispute at 0.90. My four dimensions put both under "customer protection", so the table cannot tell data privacy from a card dispute. My numbers are not wrong; the dimension set is too coarse. Real embeddings with hundreds of dimensions separate them, so I left the numbers unchanged and recorded this as the lesson.

What it means for a bank team. The closest pair says an agent asking "how do I raise a dispute?" will also be shown the complaints-escalation snippet. That is helpful, because one process often leads to the other. A human

still needs to check that the cited clause actually answers the question, because Privacy also scores 0.90 and a high score is not proof of relevance. They must also check that the snippet is the current, approved policy wording before quoting it to a customer.

Task D - Decision cases

Case	Metric and why	How I would choose the threshold	Who reviews / what is automated?
1. Knowledge-search assistant answers an agent's policy question or says "I don't know"	Cosine. This is text: the question and the policy chunks differ in length, and cosine ignores length and compares topic direction.	Build a labelled set (e.g. 100 real-style synthetic questions, each with the correct clause, plus 20 that the policy does not answer). Plot the top-1 cosine for correct, wrong and unanswerable cases. Pick an upper cut-off where answers above it are right in, say, 19 of 20 cases, and a lower cut-off below which most unanswerable questions fall. Agree both with the policy owner and re-measure after any model or content change.	Automated: retrieval, showing the cited clause, and "I don't know" below the lower cut-off. Grey band: the answer is shown marked "verify" with the citation; the agent reads the clause before telling the customer. The policy team reviews grey-band and "I don't know" logs weekly.
2. KYC tool compares an extracted address with the address on file and flags mismatches	Both, but neither alone. Cosine (on normalised address text) tolerates formatting such as "Flat 4, MG Rd" vs "4 M.G. Road", but two addresses differing only by house number or PIN code can still score 0.99. So exact field checks (PIN code, house or flat number) run first, and the score handles the free-text part.	Take labelled pairs from past remediation cases (same address written differently vs genuinely different addresses). Choose the score where false "match" decisions are close to zero, accepting more manual reviews. For identity, a missed mismatch costs far more than an extra review.	Automated: only the FLAG. Exact fields equal and very high score -> marked "consistent" (still sampled). Anything else -> KYC analyst. The tool never auto-accepts or auto-rejects a customer's identity or address, and the score, both texts and the rule are logged for audit.
3. Data pipeline suggests which two customer records may be duplicates	Both. Cosine on the text fields (name, address) catches spelling and order variants. Euclidean distance on normalised numeric fields (date-of-birth parts, account counts) catches records that are numerically almost identical.	Use a set of known duplicates and known non-duplicates, such as siblings at the same address. Measure precision for the top suggestions at several cut-offs. Choose the cut-off that keeps the review queue manageable while missing few true duplicates, and keep a combined rule, e.g. text cosine >= x AND numeric distance <= y.	Automated: ranking and suggesting pairs with their scores. Data steward: confirms every merge. Never auto-merge, because merging two real customers mixes their money and their KYC. Every suggestion and decision is logged with the score that triggered it.

Task E - Where it breaks

1. A misleading high score

```
TASK E - false similarity (dimensions: lending, fraud_risk, kyc_compliance, customer_service)
'Approve the loan'  = [5, 1, 2, 2]
'Decline the loan'  = [5, 1, 2, 1]
cosine = 0.986   distance = 1.000

From my scenario dimensions (eligibility, fees, lifecycle, protection):
'Applicant IS eligible: salary account, age 21-60'     = [5, 0, 2, 0]
'Applicant is NOT eligible: no salary account, age 19'  = [5, 0, 2, 1]
cosine = 0.983   distance = 1.000
```

Cosine above 0.98 for opposite decisions: one number differs by 1, so the arrows point almost the same way.

2. Why this happens, and the safeguard

Embedding models learn from the contexts words appear in. "Approve the loan" and "decline the loan" appear in the same documents, near the same words (application, eligibility, credit score, EMI), so their vectors land in the same neighbourhood. The single word that flips the meaning, a negation or "not", is a small part of the whole text and moves the vector only a little. Cosine measures topic direction, not truth or polarity, so it reports

"same topic" correctly but cannot say "opposite decision". The safeguard I would add in a banking process: the similarity score only retrieves and ranks. The decision text itself (approve/decline, eligible/not eligible) is checked by an explicit rule or a keyword/negation check, the cited clause is always shown, and any action affecting a customer's money or eligibility needs a person to read the source before it is used.

3. Why closer vectors mean similar meaning (my own words)

A model gives each piece of text an address on a "map of meaning". It learns those addresses by seeing which words keep the same company, so texts used in the same situations end up close together. Comparing meaning then becomes measuring how close two addresses are, either by the angle between the arrows (cosine) or by the straight-line gap (distance). An example from my IT service-desk work: "I can't log in to VPN" and "remote access is not connecting" share almost no words. They still land close together because both appear next to the same things (password, network, token, laptop), so a similarity search sends both to the network team. Being close only means "talked about in the same way". "VPN works now" would also be close, which is why a person still reads the ticket before closing it.

Stretch tasks

```
STRETCH 1 - unit vectors: distance^2 = 2 - 2 x cosine  (pair: Fees vs Early closure)
unit Fees = [0.189, 0.9449, 0.189, 0.189]
unit Early closure = [0.0, 0.6172, 0.7715, 0.1543]
distance^2 = 0.483649   2 - 2*cosine = 0.483649   equal: True

STRETCH 2 - centroid of two groups (idea behind Scenario 4 drift check)
centre of fee/closure snippets = [0.5, 4.5, 3.0, 1.0]
centre of protection snippets  = [0.33, 1.0, 1.0, 4.67]
cosine between centres = 0.457, distance = 5.452

STRETCH 3 - sensitivity: change one number by +2 in 'Fees' and compare with 'Early closure'
baseline cosine 0.758
  +2 on eligibility          -> 0.669 (-0.090)
  +2 on fees_and_charges     -> 0.728 (-0.031)
  +2 on account_lifecycle    -> 0.926 (+0.168)
  +2 on customer_protection  -> 0.720 (-0.038)
```

Reading the results. (1) The identity holds exactly for unit vectors, so for normalised embeddings cosine and distance always give the same ranking. (2) The two group centres are far apart (cosine 0.46), which is what a drift check would track over time for Scenario 4. (3) Adding 2 to account_lifecycle moved the score most (+0.168), because that is where Early closure is strongest. A change counts most on the dimensions where the other vector is large, so the dimensions are not weighted equally; their influence depends on the other vector.